

SENAI - CRTI

Apostila

Atualização e Exclusão com APIs (PUT, PATCH, DELETE)

Projeto guiado: JSONPlaceholder

Interação com APIs — Sprint 3

Sumário

- Introdução e pré-requisitos
- Conceitos básicos
- Passo a passo: criando o projeto pelo terminal
- Código-fonte completo, arquivo por arquivo
- Executando e testando a aplicação
- Testes automatizados
- Build de produção

1. Introdução e pré-requisitos

Nesta apostila você vai construir, do zero, a aplicação Angular JSONPlaceholder e evoluí-la até o ciclo CRUD completo: criar uma tarefa (Create), listar as tarefas existentes (Read), editar uma tarefa inteira ou só um campo dela (Update, com PUT e PATCH) e remover uma tarefa (Delete). Todas as operações conversam com a API pública JSONPlaceholder, que simula respostas reais de um servidor sem persistir os dados de verdade — ideal para treinar antes de ligar a uma API de produção.

Siga os capítulos na ordem. Cada comando de terminal e cada arquivo de código aparecem completos, prontos para copiar.

Pré-requisitos

- Node.js instalado (versão LTS mais recente).
- Angular CLI instalado globalmente.
- Um editor de código (ex.: VS Code).

Para instalar o Angular CLI globalmente, caso ainda não tenha:

```
$ npm install -g @angular/cli
```

Para conferir se tudo está pronto, rode os três comandos abaixo. Eles devem responder com um número de versão, sem erro:

```
$ node --version  
$ npm --version  
$ ng version
```

2. Conceitos básicos

Antes de programar, é importante entender o papel de cada peça que vamos usar.

HttpClient

É o serviço oficial do Angular para fazer requisições HTTP (GET, POST, PUT, PATCH, DELETE...). Nesta apostila usamos os métodos `get()`, `post()`, `put()`, `patch()` e `delete()` para conversar com a API.

Método GET — Listar

Busca dados já existentes no servidor, sem alterar nada. Usamos GET para carregar a lista de tarefas assim que a tela abre.

Método POST — Criar

Cria um novo recurso no servidor. O corpo (body) da requisição carrega os dados do formulário; a resposta de sucesso é 201 Created, geralmente com o id gerado para o novo recurso.

Método PUT — Atualizar tudo

Substitui o recurso inteiro no servidor: todos os campos precisam ser enviados, mesmo os que não mudaram. É um método idempotente — enviar o mesmo PUT várias vezes produz sempre o mesmo resultado. Usamos PUT quando o aluno edita a tarefa inteira pelo formulário de edição.

Método PATCH — Atualizar parte

Altera só os campos enviados; o resto do recurso permanece como estava no servidor. É a escolha certa para uma mudança pontual, como marcar a tarefa como concluída sem reenviar título, descrição e categoria. No TypeScript, o tipo `Partial<Tarefa>` descreve exatamente isso: um objeto com qualquer subconjunto dos campos de Tarefa.

Método DELETE — Remover

Remove o recurso do servidor. Como é uma ação irreversível, é boa prática confirmar com o usuário antes de disparar a requisição — nesta apostila usamos a função nativa do navegador `confirm('mensagem')`, que pausa a execução e só continua se o usuário clicar em "OK".

Payload

O payload é o corpo da requisição HTTP — o objeto de dados enviado ao servidor. No Angular, basta passar um objeto como argumento de `http.post(url, payload)`, `http.put(url, payload)` ou `http.patch(url, payload)` que o HttpClient já cuida de serializá-lo em JSON e de definir o header `Content-Type: application/json` automaticamente.

Reactive Forms (FormGroup e FormControl)

Reactive Forms é a abordagem do Angular para formulários controlados via código TypeScript. Um `FormGroup` agrupa vários `FormControl`, um para cada campo do formulário, e cada `FormControl`

pode ter validações (ex.: `Validators.required`). O método `form.getRawValue()` devolve um objeto com todos os valores atuais dos campos — exatamente o payload que enviamos para a API.

Service

Uma classe separada do componente, decorada com `@Injectable({ providedIn: 'root' })`, responsável apenas por se comunicar com a API. Isolar essa lógica em um service evita duplicar código e deixa a aplicação mais fácil de manter e testar. Todos os métodos HTTP do service devolvem um Observable — eles não disparam a requisição sozinhos, só quando alguém assina (`subscribe`) esse Observable.

Model (Interface)

Uma interface TypeScript que descreve o formato dos dados de uma tarefa (título, descrição, prioridade, categoria, status e data limite). Isso dá autocomplete e evita nomes de campo errados na hora de montar o payload.

Signals

Signals são variáveis reativas do Angular. Quando o valor de um signal muda (ex.: `signal.set(novoValor)` ou `signal.update(...)`), a tela é atualizada automaticamente, sem precisar de comandos manuais de renderização.

Observable e subscribe

O `HttpClient` não devolve o dado na hora — ele devolve um Observable, que representa uma resposta que vai chegar no futuro. Para receber esse valor quando ele chegar (ou tratar um erro), usamos `.subscribe({ next, error })`.

Zoneless (Angular 21+)

Desde a versão 21, o Angular não depende mais da biblioteca Zone.js para saber quando atualizar a tela. Em vez disso, a atualização acontece diretamente quando um Signal muda de valor. Isso deixa a aplicação mais leve e rápida. A partir do Angular 22, esse modo é habilitado explicitamente no `appConfig` com `provideZonelessChangeDetection()`.

Testando código que usa HttpClient

Quando um componente ou service faz uma chamada HTTP automaticamente (por exemplo, ao ser criado), o teste precisa simular essa chamada em vez de acessar a internet de verdade. Para isso trocamos `provideHttpClient()` por `provideHttpClient() + provideHttpClientTesting()`, e usamos um `HttpTestingController` para interceptar a requisição e devolver uma resposta falsa com `.flush(...)`.

Resumo: ação da tela → verbo HTTP → método do service

- Abrir a tela → carrega a lista → GET → `tarefaService.listar()`
- Clicar em "Salvar" (novo) → cria a tarefa → POST → `tarefaService.criar(tarefa)`
- Clicar em "Salvar alteração" (editar) → substitui a tarefa inteira → PUT → `tarefaService.atualizar(id, tarefa)`

- Clicar em "Concluir" / "Reabrir" → altera só o status → PATCH → `taskService.atualizarParcial(id, { status })`
- Clicar em "Excluir" → remove a tarefa → DELETE → `taskService.remover(id)`

3. Passo a passo: criando o projeto pelo terminal

Abra o terminal na pasta onde deseja criar o projeto e siga os comandos na ordem.

3.1 Criar o projeto Angular

O que cada opção faz: `--style=css` usa CSS puro para os estilos; `--routing=false` porque a aplicação tem uma única tela, não precisamos de rotas; `--ssr=false` porque a aplicação roda só no navegador (client-side), sem Server-Side Rendering; `--defaults` aceita as demais respostas padrão automaticamente, sem perguntas interativas (o Angular 22 já cria o projeto zoneless por padrão).

```
$ ng new JSONPlaceholder --style=css --routing=false --ssr=false --defaults
```

3.2 Entrar na pasta do projeto

```
$ cd JSONPlaceholder
```

3.3 Gerar o model (interface) de Tarefa

Esse comando cria o arquivo `src/app/models/tarefa.ts`, com uma interface `Tarefa` vazia. Vamos preencher os campos dela — e acrescentar mais uma interface, `Post`, e uma função utilitária, `paraTarefa` — no capítulo 4.

```
$ ng generate interface models/tarefa
```

3.4 Gerar o service de Tarefa

Esse comando cria dois arquivos: `src/app/services/tarefa.ts` (o service, já decorado com `@Injectable({ providedIn: 'root' })`) e `src/app/services/tarefa.spec.ts` (o arquivo de teste). Por padrão, o Angular CLI nomeia a classe gerada apenas como `Tarefa` — mas como já existe uma interface com esse mesmo nome, renomeie a classe do service para `TarefaService` (o nome do arquivo continua `tarefa.ts`, seguindo a convenção do Angular 22 de não usar mais sufixos de tipo nos nomes de arquivo).

```
$ ng generate service services/tarefa
```

3.5 Nenhum componente novo é necessário

Todo o CRUD desta apostila roda dentro do componente raiz, gerado automaticamente pelo `ng new` (`src/app/app.ts`, `app.html` e `app.css`). Como a aplicação tem uma única tela, não é preciso rodar `ng generate component` — o formulário de criação, a lista de tarefas, o modo de edição e as ações de concluir/excluir vivem todos nesse mesmo componente. Em um projeto maior, cada um desses blocos poderia virar um componente próprio.

3.6 Estrutura final de pastas

Depois de todos os comandos, e com os arquivos preenchidos (próximo capítulo), a estrutura deve ficar assim:

```
src/  
  main.ts  
  app/  
    app.config.ts  
    app.ts  
    app.html  
    app.css  
    app.spec.ts  
  models/  
    tarefa.ts  
  services/  
    tarefa.ts  
    tarefa.spec.ts
```

4. Código-fonte completo, arquivo por arquivo

A seguir está o conteúdo final de cada arquivo, já com o ciclo CRUD completo. Abra cada um no editor e substitua o conteúdo pelo código abaixo.

4.1 src/main.ts

Ponto de entrada da aplicação. Inicializa (bootstrap) o componente principal App usando as configurações definidas em appConfig. Este arquivo é gerado pronto pelo Angular CLI — não precisa alterá-lo.

Arquivo: src/main.ts

```
import { bootstrapApplication } from '@angular/platform-browser';
import { appConfig } from './app/app.config';
import { App } from './app/app';

bootstrapApplication(App, appConfig).catch((err) => console.error(err));
```

4.2 src/app/app.config.ts

Configura os providers globais da aplicação: provideZonelessChangeDetection() liga o modo zoneless explicitamente, e provideHttpClient() habilita o HttpClient — sem essa linha, o TarefaService não consegue fazer requisições HTTP.

Arquivo: src/app/app.config.ts

```
import {
  ApplicationConfig,
  provideBrowserGlobalErrorListeners,
  provideZonelessChangeDetection,
} from '@angular/core';
import { provideHttpClient } from '@angular/common/http';

export const appConfig: ApplicationConfig = {
  providers: [
    provideBrowserGlobalErrorListeners(),
    provideZonelessChangeDetection(),
    provideHttpClient(),
  ],
};
```

4.3 src/app/models/tarefa.ts

A interface Tarefa descreve o formato dos dados usados na tela (o campo id é opcional porque só existe depois que o servidor cria o recurso). A interface Post descreve o formato real que a JSONPlaceholder devolve em GET /posts (userId, id, title, body) — diferente do formato de Tarefa. A função paraTarefa converte um Post recebido da API em uma Tarefa que a tela sabe exibir.

Arquivo: src/app/models/tarefa.ts

```
export interface Tarefa {
```

```

    id?: number;
    titulo: string;
    descricao: string;
    prioridade: 'alta' | 'media' | 'baixa';
    categoria: string;
    status: 'pendente' | 'em-andamento' | 'concluida';
    dataLimite: string;
  }

  /** Formato real devolvido por GET /posts na JSONPlaceholder. */
  export interface Post {
    id: number;
    userId: number;
    title: string;
    body: string;
  }

  /** Adapta um Post da API para o formato de Tarefa usado na tela. */
  export function paraTarefa(post: Post): Tarefa {
    return {
      id: post.id,
      titulo: post.title,
      descricao: post.body,
      prioridade: 'media',
      categoria: 'geral',
      status: 'pendente',
      dataLimite: '',
    };
  }
}

```

4.4 src/app/services/tarefa.ts

O service concentra toda a comunicação com a API, um método para cada operação do CRUD. listar() faz GET e devolve os Post crus da API. criar() faz POST com uma Tarefa nova. atualizar() faz PUT, substituindo o recurso inteiro — por isso recebe uma Tarefa completa. atualizarParcial() faz PATCH e aceita Partial<Tarefa>, ou seja, qualquer subconjunto dos campos. remover() faz DELETE e não precisa de corpo na requisição.

Arquivo: src/app/services/tarefa.ts

```

import { inject, Injectable } from '@angular/core';
import { HttpClient } from '@angular/common/http';
import { Post, Tarefa } from '../models/tarefa';

@Injectable({ providedIn: 'root' })
export class TarefaService {
  private http = inject(HttpClient);
  private api = 'https://jsonplaceholder.typicode.com/posts';

  listar() {
    return this.http.get<Post[]>(`${this.api}?_limit=5&userId=1`);
  }

  criar(tarefa: Tarefa) {
    return this.http.post<Tarefa>(this.api, tarefa);
  }
}

```

```

    atualizar(id: number, tarefa: Tarefa) {
      return this.http.put<Tarefa>(`${this.api}/${id}`, tarefa);
    }

    atualizarParcial(id: number, dados: Partial<Tarefa>) {
      return this.http.patch<Tarefa>(`${this.api}/${id}`, dados);
    }

    remover(id: number) {
      return this.http.delete(`${this.api}/${id}`);
    }
  }
}

```

4.5 src/app/app.ts

O componente principal. Mantém dois FormGroup — form para criar uma tarefa nova e formEdicao para editar uma já existente — e o estado da tela em signals (tarefas, carregando, enviando, erro, idEmEdicao). No construtor, chama carregar() para buscar a lista assim que a tela abre. Cada ação da tela (salvar, iniciarEdicao/salvarEdicao, alternarStatus, excluir) chama o método correspondente do service e, no next do subscribe, atualiza o signal tarefas — é essa atualização que a tela reflete automaticamente, graças aos signals.

Arquivo: src/app/app.ts

```

import { Component, inject, signal } from '@angular/core';
import { FormControl, FormGroup, ReactiveFormsModule, Validators } from
'@angular/forms';
import { TarefaService } from '../services/tarefa';
import { paraTarefa, Tarefa } from '../models/tarefa';

@Component({
  selector: 'app-root',
  imports: [ReactiveFormsModule],
  templateUrl: './app.html',
  styleUrls: ['./app.css'],
})
export class App {
  private tarefaService = inject(TarefaService);

  form = new FormGroup({
    titulo: new FormControl('', { nullable: true, validators:
[Validators.required] }),
    descricao: new FormControl('', { nullable: true, validators:
[Validators.required] }),
    prioridade: new FormControl<Tarefa['prioridade']>('media', { nullable: true
}),
    categoria: new FormControl('', { nullable: true, validators:
[Validators.required] }),
    status: new FormControl<Tarefa['status']>('pendente', { nullable: true }),
    dataLimite: new FormControl('', { nullable: true, validators:
[Validators.required] }),
  });

  formEdicao = new FormGroup({

```

```

        titulo: new FormControl('', { nullable: true, validators:
[Validators.required] })),
        descricao: new FormControl('', { nullable: true, validators:
[Validators.required] })),
        prioridade: new FormControl<Tarefa['prioridade']>('media', { nullable: true
}),
        categoria: new FormControl('', { nullable: true, validators:
[Validators.required] })),
        dataLimite: new FormControl('', { nullable: true })),
    });

    tarefas = signal<Tarefa[]>([]);
    carregando = signal(false);
    enviando = signal(false);
    erro = signal('');

    idEmEdicao = signal<number | null>(null);

    // A JSONPlaceholder sempre responde ao POST com o mesmo id (101), então geramos
    // um id local exclusivo para cada tarefa criada nesta tela, evitando que duas
    // tarefas criadas colidam com o mesmo id (o que quebra o "track" da lista).
    private proximoIdLocal = Date.now();

    constructor() {
        this.carregar();
    }

    carregar() {
        this.carregando.set(true);
        this.erro.set('');

        this.tarefaService.listar().subscribe({
            next: (posts) => {
                this.tarefas.set(posts.map(paraTarefa));
                this.carregando.set(false);
            },
            error: () => {
                this.erro.set('Erro ao carregar as tarefas. Tente novamente.');
```

```

        this.tarefas.update((lista) => [tarefaCriada, ...lista]);
        this.form.reset({ prioridade: 'media', status: 'pendente' });
    },
    error: () => {
        this.enviando.set(false);
        this.erro.set('Erro ao criar a tarefa. Tente novamente.');
```

```

    },
  });
}

```

```

iniciarEdicao(tarefa: Tarefa) {
  this.idEmEdicao.set(tarefa.id ?? null);
  this.formEdicao.setValue({
    titulo: tarefa.titulo,
    descricao: tarefa.descricao,
    prioridade: tarefa.prioridade,
    categoria: tarefa.categoria,
    dataLimite: tarefa.dataLimite,
  });
}

```

```

cancelarEdicao() {
  this.idEmEdicao.set(null);
}

```

```

salvarEdicao(tarefaAtual: Tarefa) {
  if (!tarefaAtual.id || this.formEdicao.invalid) {
    this.formEdicao.markAllAsTouched();
    this.erro.set('Preencha todos os campos obrigatórios da edição.');
```

```

    return;
  }

```

```

  const id = tarefaAtual.id;
  const tarefaCompleta: Tarefa = {
    ...tarefaAtual,
    ...this.formEdicao.getRawValue(),
  };

```

```

  this.erro.set('');

```

```

  // PUT: substitui o recurso inteiro, por isso enviamos todos os campos.

```

```

  this.tarefaService.atualizar(id, tarefaCompleta).subscribe({
    next: () => this.aplicarEdicaoLocal(id, tarefaCompleta),
    error: () => {
      // A JSONPlaceholder só possui 100 posts de verdade: tarefas criadas nesta
      // tela (id > 100) não existem no servidor fake, então o PUT sempre
      // responde com erro. Como nada é persistido de fato por essa API de
      // testes, aplicamos a edição localmente mesmo assim.
      if (id > 100) {
        this.aplicarEdicaoLocal(id, tarefaCompleta);
      } else {
        this.erro.set('Erro ao atualizar a tarefa. Tente novamente.');
```

```

      }
    },
  });
}

```

```

private aplicarEdicaoLocal(id: number, tarefaCompleta: Tarefa) {
  this.tarefas.update((lista) => lista.map((t) => (t.id === id ? tarefaCompleta :
t)));
}

```

```

    this.idEmEdicao.set(null);
  }

  alternarStatus(tarefa: Tarefa) {
    if (!tarefa.id) return;

    const novoStatus: Tarefa['status'] =
      tarefa.status === 'concluida' ? 'pendente' : 'concluida';

    // PATCH: altera somente o campo status, preservando o restante do recurso.
    this.tarefaService.atualizarParcial(tarefa.id, { status: novoStatus
  }).subscribe({
    next: () => {
      this.tarefas.update((lista) =>
        lista.map((t) => (t.id === tarefa.id ? { ...t, status: novoStatus } :
t))),
      );
    },
    error: () => this.erro.set('Erro ao atualizar o status. Tente novamente.'),
  });
}

excluir(id?: number) {
  if (!id) return;
  if (!confirm('Tem certeza que deseja excluir esta tarefa?')) return;

  this.tarefaService.remover(id).subscribe({
    next: () => this.tarefas.update((lista) => lista.filter((t) => t.id !== id)),
    error: () => this.erro.set('Erro ao excluir a tarefa. Tente novamente.'),
  });
}
}

```

4.6 src/app/app.html

Template da tela. No topo, o formulário reativo de criação (ligado a form). Em seguida, a lista de tarefas: cada tarefa mostra, com @if, o modo de edição (ligado a formEdicao, com o botão "Salvar alteração (PUT)") quando idEmEdicao() é igual ao id daquela tarefa, ou o modo normal — título, descrição, selo de status e os botões Editar, Concluir/Reabrir e Excluir. A diretiva @for percorre o signal tarefas() e reconstrói a lista automaticamente sempre que ele muda.

Arquivo: src/app/app.html

```

<div class="container">
  <h1>Gerenciador de Tarefas · JSONPlaceholder</h1>

  <form class="form" [formGroup]="form" (ngSubmit)="salvar()">
    <label>
      Título
      <input type="text" formControlName="titulo" placeholder="Nome da tarefa" />
    </label>

    <label>
      Descrição
      <textarea formControlName="descricao" placeholder="Detalhes da
atividade"></textarea>

```

```

</label>

<div class="linha">
  <label>
    Prioridade
    <select formControlName="prioridade">
      <option value="baixa">Baixa</option>
      <option value="media">Média</option>
      <option value="alta">Alta</option>
    </select>
  </label>

  <label>
    Categoria
    <input type="text" formControlName="categoria" placeholder="Classificação
da tarefa" />
  </label>
</div>

<div class="linha">
  <label>
    Status
    <select formControlName="status">
      <option value="pendente">Pendente</option>
      <option value="em-andamento">Em andamento</option>
      <option value="concluida">Concluída</option>
    </select>
  </label>

  <label>
    Data limite
    <input type="date" formControlName="dataLimite" />
  </label>
</div>

<button type="submit" [disabled]="enviando()">
  {{ enviando() ? 'Salvando...' : 'Salvar' }}
</button>
</form>

@if (erro()) {
  <p class="erro">{{ erro() }}</p>
}

<div class="lista">
  <h2>Tarefas</h2>

  @if (carregando()) {
    <p class="carregando">Carregando tarefas...</p>
  }

  @if (!carregando() && tarefas().length === 0) {
    <p class="vazio">Nenhuma tarefa cadastrada ainda.</p>
  }

  @for (tarefa of tarefas(); track tarefa.id) {
    <div class="card">
      @if (idEmEdicao() === tarefa.id) {

```

```

        <form class="form form-edicao" [formGroup]="formEdicao"
        (ngSubmit)="salvarEdicao(tarefa)">
            <label>
                Título
                <input type="text" formControlName="titulo" />
            </label>

            <label>
                Descrição
                <textarea formControlName="descricao"></textarea>
            </label>

            <div class="linha">
                <label>
                    Prioridade
                    <select formControlName="prioridade">
                        <option value="baixa">Baixa</option>
                        <option value="media">Média</option>
                        <option value="alta">Alta</option>
                    </select>
                </label>

                <label>
                    Categoria
                    <input type="text" formControlName="categoria" />
                </label>
            </div>

            <label>
                Data limite
                <input type="date" formControlName="dataLimite" />
            </label>

            <div class="acoes">
                <button type="submit">Salvar alteração (PUT)</button>
                <button type="button" class="btn-secundario"
                (click)="cancelarEdicao()">Cancelar</button>
            </div>
        </form>
    } @else {
        <div class="card-topo">
            <strong>{{ tarefa.titulo }}</strong>
            <span class="badge" [class.badge-concluida]="tarefa.status ===
            'concluida'">
                {{ tarefa.status }}
            </span>
        </div>
        <p>{{ tarefa.descricao }}</p>
        <p class="meta">
            {{ tarefa.categoria }} · prioridade {{ tarefa.prioridade }}
            @if (tarefa.dataLimite) {
                · até {{ tarefa.dataLimite }}
            }
        </p>

        <div class="acoes">
            <button type="button" class="btn-secundario"
            (click)="iniciarEdicao(tarefa)">
                Editar
            </button>
        </div>
    }
}

```

```

        <button type="button" class="btn-secundario"
(click)="alternarStatus(tarefa)">
            {{ tarefa.status === 'concluida' ? 'Reabrir' : 'Concluir' }}
        </button>
        <button type="button" class="btn-perigo"
(click)="excluir(tarefa.id)">Excluir</button>
    </div>
}
</div>
}
</div>
</div>

```

4.7 src/app/app.css

Estilos da tela: layout centralizado, cartão branco do formulário, grade de dois campos por linha, cartões da lista de tarefas com selo de status (amarelo para pendente/em andamento, verde para concluída) e a barra de ações (Editar, Concluir/Reabrir em cinza, Excluir em vermelho).

Arquivo: src/app/app.css

```

* {
  box-sizing: border-box;
  margin: 0;
  padding: 0;
  font-family: Arial, sans-serif;
}

:host {
  display: block;
  background: #f5f7fb;
  min-height: 100vh;
}

.container {
  width: 100%;
  max-width: 700px;
  margin: 60px auto;
  padding: 30px;
}

h1 {
  text-align: center;
  margin-bottom: 30px;
  color: #222;
}

.form {
  display: flex;
  flex-direction: column;
  gap: 16px;
  background: white;
  padding: 25px;
  border-radius: 12px;
  box-shadow: 0 4px 18px rgba(0, 0, 0, 0.08);
  margin-bottom: 25px;
}

```

```
label {
  display: flex;
  flex-direction: column;
  gap: 6px;
  font-size: 14px;
  color: #444;
  font-weight: bold;
}

.linha {
  display: flex;
  gap: 16px;
}

.linha label {
  flex: 1;
}

input,
select,
textarea {
  padding: 12px;
  border: 1px solid #ccc;
  border-radius: 8px;
  font-size: 15px;
  font-family: inherit;
  font-weight: normal;
}

textarea {
  resize: vertical;
  min-height: 70px;
}

button {
  border: none;
  background: #2563eb;
  color: white;
  padding: 14px 24px;
  border-radius: 8px;
  cursor: pointer;
  font-weight: bold;
  font-size: 15px;
}

button:hover:not(:disabled) {
  background: #1d4ed8;
}

button:disabled {
  background: #93c5fd;
  cursor: not-allowed;
}

.erro {
  color: #dc2626;
  margin-bottom: 20px;
  font-weight: bold;
}
```

```
}

.lista h2 {
  margin-bottom: 15px;
  color: #111;
}

.card {
  background: white;
  padding: 20px;
  border-radius: 12px;
  box-shadow: 0 4px 18px rgba(0, 0, 0, 0.08);
  margin-bottom: 16px;
}

.card-topo {
  display: flex;
  justify-content: space-between;
  align-items: center;
  margin-bottom: 8px;
}

.badge {
  background: #fef9c3;
  color: #854d0e;
  font-size: 12px;
  font-weight: bold;
  padding: 4px 10px;
  border-radius: 999px;
  text-transform: capitalize;
}

.badge-concluida {
  background: #dcfce7;
  color: #15803d;
}

.card p {
  color: #444;
  margin: 4px 0;
}

.meta {
  font-size: 13px;
  color: #777;
}

.carregando,
.vazio {
  color: #666;
  margin-bottom: 16px;
}

.acoes {
  display: flex;
  gap: 10px;
  margin-top: 14px;
}
```

```
.btn-secundario {  
  background: #e5e7eb;  
  color: #1f2937;  
}  
  
.btn-secundario:hover:not(:disabled) {  
  background: #d1d5db;  
}  
  
.btn-perigo {  
  background: #fee2e2;  
  color: #b91c1c;  
}  
  
.btn-perigo:hover:not(:disabled) {  
  background: #fecaca;  
}  
  
.acoes button {  
  padding: 8px 16px;  
  font-size: 13px;  
}  
  
.form-edicao {  
  margin-bottom: 0;  
  box-shadow: none;  
  padding: 0;  
}
```

5. Executando e testando a aplicação

Com todos os arquivos salvos, suba o servidor de desenvolvimento:

```
$ npm start
```

Ou, de forma equivalente:

```
$ ng serve
```

Abra o navegador em <http://localhost:4200> e teste os cenários abaixo.

Roteiro de teste

- Ao abrir a página, a lista de tarefas já vem preenchida — é o GET automático disparado no construtor do componente.
- Preencha o formulário do topo e clique em Salvar → a nova tarefa aparece no topo da lista (POST) e o formulário limpa.
- Deixe um campo obrigatório em branco e clique em Salvar → aparece "Preencha todos os campos obrigatórios.", sem nenhuma requisição sendo enviada.
- Clique em "Concluir" em uma tarefa → o selo muda para "concluída" na hora e o botão vira "Reabrir" (PATCH, só o campo status).
- Clique em "Editar" → aparece um formulário preenchido com os dados da tarefa; altere um campo e clique em "Salvar alteração (PUT)" → o cartão é atualizado com os novos valores.
- Clique em "Cancelar" durante a edição → o formulário de edição fecha sem enviar nada.
- Clique em "Excluir" → o navegador pede confirmação; ao confirmar, o cartão desaparece da lista (DELETE).

6. Testes automatizados

O Angular 22 usa o Vitest como executor de testes por padrão. Como o App agora dispara uma requisição HTTP assim que é criado (o `carregar()` do construtor), o teste de `app.spec.ts` precisa de um `HttpTestingController` para interceptar essa chamada e responder com dados falsos, em vez de acessar a internet de verdade.

6.1 `src/app/app.spec.ts`

`provideHttpClientTesting()` substitui o backend HTTP real por um simulado. `httpMock.expectOne(...)` captura a requisição de GET disparada no construtor e `.flush([])` responde com uma lista vazia, permitindo que o teste continue sem depender da rede. `httpMock.verify()`, no `afterEach`, garante que nenhuma requisição inesperada ficou sem resposta.

Arquivo: `src/app/app.spec.ts`

```
import { TestBed } from '@angular/core/testing';
import { provideHttpClient } from '@angular/common/http';
import { HttpTestingController, provideHttpClientTesting } from
 '@angular/common/http/testing';
import { App } from './app';

describe('App', () => {
  let httpMock: HttpTestingController;

  beforeEach(async () => {
    await TestBed.configureTestingModule({
      imports: [App],
      providers: [provideHttpClient(), provideHttpClientTesting()],
    }).compileComponents();

    httpMock = TestBed.inject(HttpTestingController);
  });

  afterEach(() => {
    httpMock.verify();
  });

  it('should create the app', () => {
    const fixture = TestBed.createComponent(App);
    const app = fixture.componentInstance;
    expect(app).toBeTruthy();

    httpMock.expectOne((req) =>
req.url.startsWith('https://jsonplaceholder.typicode.com/posts'))
.flush([]);
  });

  it('should render title', async () => {
    const fixture = TestBed.createComponent(App);
    httpMock.expectOne((req) =>
req.url.startsWith('https://jsonplaceholder.typicode.com/posts'))
.flush([]);
    await fixture.whenStable();

    const compiled = fixture.nativeElement as HTMLElement;
    expect(compiled.querySelector('h1')?.textContent).toContain('Gerenciador de
Tarefas');
```

```
});  
});
```

6.2 src/app/services/tarefa.spec.ts

Este teste só verifica se o service é criado corretamente pela injeção de dependência do Angular — nenhum dos seus métodos é chamado aqui, então não é preciso simular requisições.

Arquivo: src/app/services/tarefa.spec.ts

```
import { TestBed } from '@angular/core/testing';  
import { provideHttpClient } from '@angular/common/http';  
import { TarefaService } from './tarefa';  
  
describe('TarefaService', () => {  
  let service: TarefaService;  
  
  beforeEach(() => {  
    TestBed.configureTestingModule({  
      providers: [provideHttpClient()],  
    });  
    service = TestBed.inject(TarefaService);  
  });  
  
  it('should be created', () => {  
    expect(service).toBeTruthy();  
  });  
});
```

6.3 Rodando os testes

```
$ ng test --watch=false
```

O resultado esperado é a mensagem de todos os testes passando ("Tests ... passed").

7. Build de produção

Quando o projeto estiver pronto para publicar, gere a versão otimizada com:

```
$ ng build
```

Os arquivos finais são gerados na pasta `dist/JSONPlaceholder`, prontos para hospedar em qualquer servidor de arquivos estáticos.